Data Science @UL-FRI

Homework 4.1 Report

Tjaš Ajdovec

ta5946@fri.uni-lj.si

Implementing Nelder-Mead Method

The Nelder-Mead method is a **derivative-free** optimisation algorithm. It maintains a *simplex* of $n+1$ points in $\mathbb{R}^n$ and iteratively replaces the worst point with a better one. Since it only needs function values, it works on nonsmooth, noisy, or black-box objectives where gradients are unavailable.

Each iteration sorts the simplex so that $f(x_1) \leq \cdots \leq f(x_{n+1})$ and computes the centroid $\bar{x}$ over all but the worst point. The following steps are attempted in order, each replacing the worst point if successful.

1. **Reflection.** $x_r = 2\bar{x} - x_{n+1}$. Accept if $f(x_r)$ lies between the best and second worst values.
2. **Expansion.** If $f(x_r) < f(x_1)$, try $x_e = 2x_r - \bar{x}$. Accept whichever of x_e and x_r is lower.
3. **Outside contraction.** If x_r is better than the worst but not the second worst, try $x_c = (\bar{x} + x_r)/2$. Accept if $f(x_c) \leq f(x_r)$, otherwise shrink.
4. **Inside contraction.** If x_r is worse than the worst, try $x_c = (\bar{x} + x_{n+1})/2$. Accept if $f(x_c) < f(x_{n+1})$, otherwise shrink.
5. **Shrink.** Pull all points except the best halfway toward it: $x_i \leftarrow (x_1 + x_i)/2$.

The standard parameters are $\alpha = 1$ (reflection), $\gamma = 2$ (expansion), $\rho = 0.5$ (contraction) and $\sigma = 0.5$ (shrink). The algorithm stops when the function value spread across the simplex drops below τ, i.e. $f(x_{n+1}) - f(x_1) < \tau$.

The homework asks for explicit 2D and 3D versions (`nelder_mead_2d`, `nelder_mead_3d`), but there is no reason to fix the dimension. The general `nelder_mead` infers n from `x0` and builds the initial simplex from `x0` plus n axis-aligned shifts of size `diameter`, giving $n+1$ vertices in total.

Comparing Nelder-Mead with Gradient Descent Methods

Test functions and starting points

We compared Nelder-Mead (NM) against gradient descent (GD), Polyak heavy ball, Nesterov accelerated gradient, AdaGrad, Newton's method and BFGS on three functions from homework GD.2.

- A quadratic in three variables:

$$f_a(x,y,z) = (x-z)^2 + (2y+z)^2 + (4x-2y+z)^2 + x + y.$$

  The Hessian is constant, so the exact minimum can be found by solving a linear system. The optimum is $x^* \approx (-0.167, -0.229, 0.167)$ with $f^* \approx -0.198$. Starting points: $(0,0,0)$ and $(1,1,0)$.

- A Rosenbrock-like function in three variables with a curved, narrow valley:

$$f_b(x,y,z) = (x-1)^2 + (y-1)^2 + 100(y-x^2)^2 + 100(z-y^2)^2.$$

  Global minimum at $(1,1,1)$ with $f = 0$. Starting points: $(1.2, 1.2, 1.2)$ and $(-1, 1.2, 1.2)$.

- The Beale function in two variables:

$$f_c(x,y) = (1.5 - x + xy)^2 + (2.25 - x + xy^2)^2 + (2.625 - x + xy^3)^2.$$

  Global minimum at $(3, 0.5)$ with $f = 0$. Starting points: $(1,1)$ and $(4.5, 4.5)$.

Heatmaps of all three functions are shown in Figure 1, with the known minimum marked by a white cross and starting points by black crosses.

Methods and parameters

All gradient-based methods run for at most 1000 iterations and stop when the gradient norm drops below 10^{-8}. Default learning rates are $\alpha = 0.001$ for GD, Polyak and Nesterov, and $\alpha = 0.01$ for AdaGrad. Newton uses a unit step. BFGS uses a backtracking Armijo line search ($c_1 = 10^{-4}$, backtrack factor 0.5). Nelder-Mead uses default diameter $d = 1.0$ and tolerance 10^{-8}.

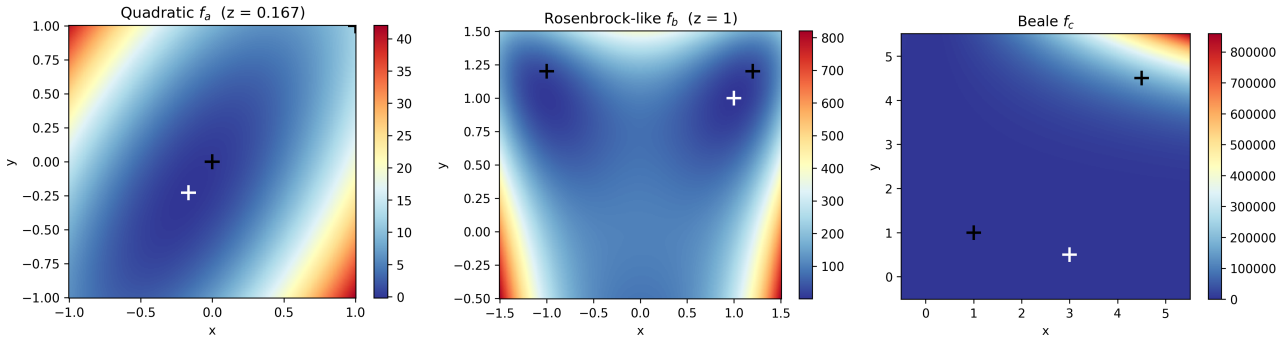

Figure 1. Heatmaps of f_a (left, z fixed at the optimum), f_b (centre, $z = 1$) and f_c (right). White cross: minimum; black crosses: starting points.

Comparison criteria

We compared the methods on six criteria [1, 2].

1. **Information used.** GD, Polyak, Nesterov and AdaGrad require only function values and gradients. Newton additionally needs the exact Hessian. BFGS builds an approximate Hessian from rank 2 updates. **Nelder-Mead uses only function values** and is the only method applicable in black-box settings.

2. **Convergence speed.** Among methods that find the correct minimum, Newton converges in at most 33 iterations, BFGS in 7–53, and Nelder-Mead in 40–227. Polyak stops early on f_a (354 and 368 iterations) but hits the 1000-iteration limit elsewhere. GD, Nesterov and AdaGrad always hit the limit.

3. **Robustness.** Nelder-Mead finds the correct minimum in **all 6 out of 6** problem/starting point combinations. BFGS and Polyak succeed on 5 out of 6, Newton on 4 out of 6, Nesterov on 3 out of 6, GD on 2 out of 6 and AdaGrad on 1 out of 6.

4. **Solution quality.** Newton and BFGS achieve $f - f^* < 10^{-16}$ on all problems where they succeed. Nelder-Mead reaches $f - f^* < 2 \times 10^{-8}$ across all six cases. GD reaches at best $f - f^* \approx 4 \times 10^{-4}$.

5. **Sensitivity to starting point.** The starting point matters most on f_c: Newton fails from both points, GD, AdaGrad, Polyak and Nesterov fail from $(4.5, 4.5)$, while Nelder-Mead succeeds from both. On f_a and f_b most methods are unaffected.

6. **Sensitivity to parameters.** GD, Polyak and Nesterov overflow on f_b at learning rate 0.01. AdaGrad fails across all tested rates on f_b and f_c. The tested learning rates were $\alpha \in \{0.0001, 0.001, 0.01\}$ and simplex diameters $d \in \{0.1, 1.0, 3.0\}$. Nelder-Mead finds the correct minimum on every problem for all tested diameters.

Numerical results

Tables 1–3 show iterations and the final gap $f(x_{\text{final}}) - f^*$ for each method and starting point. The Found column indicates whether the method found the correct minimum (✓) or not (×). A method receives × if it diverged, overflowed, or got stuck at a saddle point or wrong local minimum. Formally, Found = ✓ if and only if $\|x_{\text{final}} - x^*\| < 0.1$.

Table 1. Results on f_a. SP1 = $(0, 0, 0)$, SP2 = $(1, 1, 0)$.

	SP1			SP2		
Method	Found	Iter	Gap	Found	Iter	Gap
GD	✓	1000	4.19×10^{-4}	✓	1000	6.35×10^{-3}
Polyak	✓	354	2.78×10^{-17}	✓	368	0
Nesterov	✓	1000	1.10×10^{-8}	✓	1000	1.68×10^{-7}
AdaGrad	✓	1000	1.49×10^{-8}	×	1000	2.52
Newton	✓	1	0	✓	1	2.78×10^{-17}
BFGS	✓	7	0	✓	8	0
NM	✓	60	2.48×10^{-10}	✓	62	3.58×10^{-9}

Table 2. Results on f_b. SP1 = $(1.2, 1.2, 1.2)$, SP2 = $(-1, 1.2, 1.2)$.

	SP1			SP2		
Method	Found	Iter	Gap	Found	Iter	Gap
GD	×	1000	8.14×10^{-3}	×	1000	3.44
Polyak	✓	1000	8.93×10^{-8}	✓	1000	3.67×10^{-5}
Nesterov	×	1000	NaN	×	1000	NaN
AdaGrad	×	1000	2.39×10^{-2}	×	1000	4.22
Newton	✓	8	6.71×10^{-25}	✓	33	2.99×10^{-19}
BFGS	✓	23	1.32×10^{-20}	✓	53	1.01×10^{-20}
NM	✓	90	1.05×10^{-9}	✓	227	3.41×10^{-9}

Table 3. Results on f_c. SP1 = $(1, 1)$, SP2 = $(4.5, 4.5)$.

	SP1			SP2		
Method	Found	Iter	Gap	Found	Iter	Gap
GD	×	1000	8.09×10^{-2}	×	1000	NaN
Polyak	✓	1000	4.00×10^{-5}	×	1000	NaN
Nesterov	✓	1000	1.36×10^{-5}	×	1000	NaN
AdaGrad	×	1000	2.53	×	1000	6.55×10^{4}
Newton	×	1	14.2	×	13	14.2
BFGS	✓	15	6.09×10^{-18}	×	1000	7.31
NM	✓	40	2.36×10^{-9}	✓	40	1.46×10^{-8}

Black Box Optimization

In a **black-box setting** the objective is available only as an executable: given a point it returns a scalar after a slow computation, with no access to gradients or source code. The three functions $f_{63210005,1}, f_{63210005,2}, f_{63210005,3} : \mathbb{R}^3 \to \mathbb{R}$ are provided via `hw_4_1_win.exe`, which takes the student ID, function index and three coordinates as input. Student ID: `63210005`.

We applied two methods, both starting from $x_0 = (0,0,0)$ with a 120-second time limit per function.

Nelder-Mead (`nelder_mead_bb`) is used directly with function value caching to avoid redundant calls. Each iteration costs at most two evaluations, so many iterations fit within the budget.

BFGS with finite-difference gradients (`bfgs_bb`) approximates each partial derivative with central differences:

$$\frac{\partial f}{\partial x_j}(x) \approx \frac{f(x+he_j) - f(x-he_j)}{2h}, \quad h = 10^{-5}.$$

This costs $2n = 6$ evaluations per gradient. BFGS handles approximate gradients well, making it the most suitable gradient-based method for this setting [2]. The gradient is computed sequentially to keep the call count comparable with Nelder-Mead. Both methods use tolerance $\tau = 10^{-12}$.

Since the time limit check fires at the start of each iteration, the last iteration always completes. Actual elapsed time was 130–160 s despite the 120 s limit.

Table 4. Black-box optimisation results. Reported value is $f(x^*)$ at the best point found within the time limit.

Method	f_1	f_2	f_3
NM	0.500075433797	**0.660268107497**	0.500043678175
BFGS	0.500012360000	3.075453456212	0.500012422138

On f_1 and f_3 both methods find the same basin with comparable values. On f_2 **BFGS completes only 3 iterations**: the finite-difference gradients are misleading for f_2, causing around 9 Armijo halvings per step, so nearly all evaluations are spent on the line search. Nelder-Mead is unaffected and reaches a substantially better value.

A Toy Solution to a Linear Problem

We solved the following linear programme:

$$\begin{aligned} \max \quad & x_1 + 3x_2 + 4x_3 \\ \text{s.t.} \quad & x_1 - x_2 + x_3 \le 5, \\ & x_1 + x_2 - x_3 \le 7, \\ & -x_1 + x_2 + x_3 \le 10, \\ & x_1 + 2x_2 + 3x_3 \le 8, \\ & 3x_1 + 2x_2 + x_3 \le 6, \\ & x_1, x_2, x_3 \ge 0. \end{aligned}$$

There are eight inequalities in total: five structural constraints plus three non-negativity constraints, all treated identically. Every vertex of the feasible polytope in $\mathbb{R}^3$ is determined by exactly three active constraints. We therefore enumerated all $\binom{8}{3} = 56$ triples Θ, solved each as a 3×3 linear system to get a candidate vertex x^Θ, kept only feasible ones and picked the one maximising $c^\top x$.

The **optimal solution is** $x^* = (0, 2.5, 1)$ with objective value $f(x^*) = 11.5$. The three active constraints at the optimum are:

$$x_1 + 2x_2 + 3x_3 = 8, \quad 3x_1 + 2x_2 + x_3 = 6, \quad x_1 = 0.$$

This approach does not scale well. With n variables and m constraints the number of triples grows as $\binom{m}{n}$, which for $n = 10$, $m = 50$ is already $\binom{50}{10} \approx 10^{10}$. For larger problems, the simplex method or interior-point methods [1, 2] are used instead.

References

[1] Kurt Mehlhorn and Sanjeev Saxena. A still simpler way of introducing the interior-point method for linear programming, 2015.

[2] Imre Pólik and Tamás Terlaky. Interior point methods for nonlinear optimization. In Gianni Di Pillo and Fabio Schoen, editors, *Nonlinear Optimization*, volume 1989 of *Lecture Notes in Mathematics*, pages 215–276. Springer, Berlin, Heidelberg, 2010.